EE 604T: Pattern Recognition and Machine Learning

Course Project Title: Loan Risk Prediction

Submitted by:

Abhinandan Jain, CS23BT012

Course Instructor:

Dr. Rahul Pandya

Outline

Introduction

Methodology

Experimental Setup

Results

Conclusion

Introduction

In this project, we have explored and compared various Machine Learning models for three tasks: **Classification, Regression, and Clustering**. The data set used contains features that describe the financial situation and personal attributes of individuals. In **Classification**, the goal is to predict whether a loan is approved or not. For the **Regression** task, we use models to estimate an individual's risk score. **Clustering** involves identifying clusters in the data, where we use techniques such as the ***Elbow Method*** and ***Silhouette Analysis*** to get the optimal number of clusters.

Introduction

We use a synthetic dataset comprising of 20,000 records of personal and financial data.

The dataset contains the following thirty-six columns which can be broadly classified into:

- Demographic and Employment Information
- Financial and Credit Information
- Credit History and Loan Information
- Loan Attributes
- Target Variables

Methodology

Supervised Learning:
Classification for LoanApproved:

- KNN (with and without Mahalanobis)
- Logistic Regression
- SVM (RBF + Poly)
- Neural Network

Supervised Learning:
Regression for RiskScore:

- Linear and Polynomial Regression
- Random Forest Regressor
- XGBoost
- Neural Networks

Unsupervised Learning:

- K-Medoid
- DBSCAN
- Hierarchical Clustering

Experimental Setup

Columns with multiple classes: The columns with multiple classes as strings were converted to ***integer labels*** and ***one-hot encodings*** in separate datasets.

Duplicate and Missing Values: No duplicate rows or rows with missing values were found.

Train-Validation-Test Split: We used a 60-20-20 split which resulted in a training set with 12000 examples and validation and test sets with 4000 examples each.

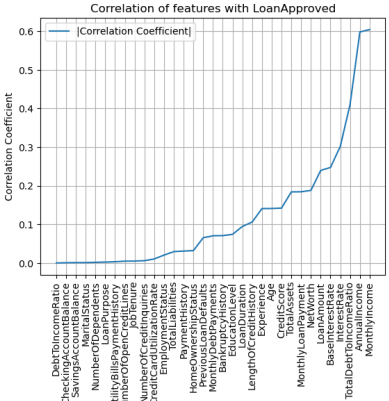
Outlier Processing: The boxplot showed presence of many outliers but they were not removed since they indicate valid data points. Hence, we used sklearn's RobustScaler which scales using IQR and median.

Dimensional Reduction: The column ApplicationDate was dropped since it does not affect LoanApproved or RiskScore. ***Principal Component Analysis*** has been used to further reduce the dimensions of the data. For data with integer labels, dimensions have been reduced from 33 to 10, 15, 20, and 25. For data with one-hot encoding, dimensions have been reduced from 49 to 15, 20, 25, 30, and 35.

Result

Classification:

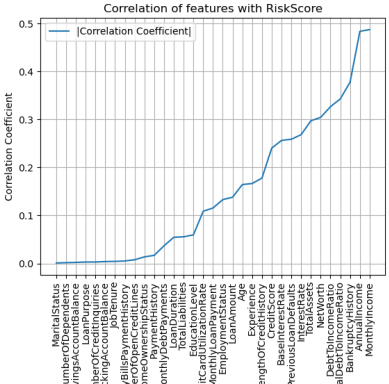
- Analysis of correlation of features with LoanApproved.



Result

Regression:

- Analysis of correlation of features with RiskScore.



Result

Classification:

KNN

Using sklearn.neighbors' KNeighborsClassifier with $k = 3, 5, 7, 11$ and *Mahalanobis* Variant, we got best **accuracy of 82.50%** and **precision of 81.84%** at $k = 11$, best **recall of 44.35%** and best **F1-score of 52.65%** on test data.

Best Performing Dataset was with:

- One-Hot Encoding
- Z-score normalization

The test Performance metrics are:

- Accuracy: 82.50%
- Precision: 81.84%
- Recall: 34.41%
- F1-Score: 48.45%

Result

Classification:

Logistic Regression

We used the sklearn LogisticRegression model with `max_iter=1000` across different preprocessed datasets. Best precision of 93.98% was obtained on the min-max scaled data with one-hot encoding and best recall of 92.99% was obtained on RobustScaled data with one-hot encoding.

Best Performing Dataset was with:

- One-Hot Encoding
- Z-score normalization

The test Performance metrics are:

- Accuracy: 96.70%
- Precision: 93.19%
- Recall: 92.99%
- F1-Score: 93.09%

Result

Classification:

SVM (RBF)

The sklearn.svm's SVC was used with `kernel='rbf'`, $C = 0.1, 1, 10$ and `gamma='scale'`. Best results were obtained with $C = 1$.

Best Performing Dataset was with:

- One-Hot Encoding
- Z-score normalization

The test Performance metrics are:

- Accuracy: 96.20%
- Precision: 93.23%
- Recall: 90.69%
- F1-Score: 91.94%

Result

Classification:

SVM (Polynomial)

The sklearn.svm's SVC was used with `kernel='poly'`, $C = 0.1, 1, 10$ and `gamma='scale'`. Best results were obtained with $C = 1$.

Best Performing Dataset was with:

- One-Hot Encoding
- Z-score normalization

The test Performance metrics are:

- Accuracy: 94.20%
- Precision: 94.80%
- Recall: 80.13%
- F1-Score: 86.85%

Result

Classification:

Neural Network

We used torch.nn's neural network along with loss criterion of BCELoss and optimizer Adam. The number of hidden layers were varied from 1 to 3 and the number of neurons in hidden layers were varied as 8, 16, 32, 64, and 128.

Best Performing Dataset was with (Triple Layer Architecture with 8, 32, 64 hidden neurons):

- One-Hot Encoding
- Z-score normalization

The test Performance metrics are:

- Accuracy: 96.77%
- Precision: 93.21%
- Recall: 93.31%
- F1-Score: 93.26%

Result

Overall Performance:

One-Hot encoded and Z-score normalized data gave the best performance across almost all the models. Logistic Regression and Triple Layer Neural Network models gave the best performance.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	96.70%	93.19%	92.99%	93.09%
Artificial Neural Network	96.77%	93.21%	93.31%	93.26%

Result

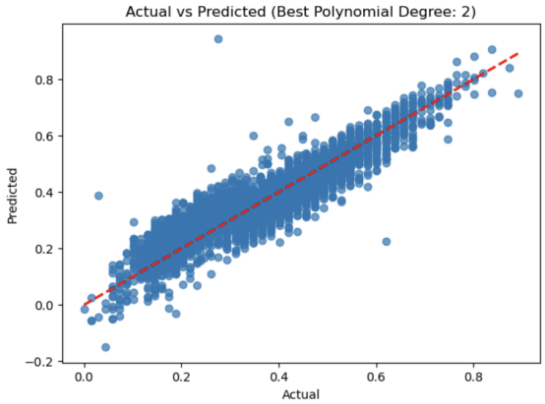
Regression:

Polynomial Regression

The target variable RiskScore is the last column in the preprocessed datasets. The test RMSE is 0.066.

Best Performing Dataset was with:

- One-Hot Encoding
- Min-Max normalization



Result

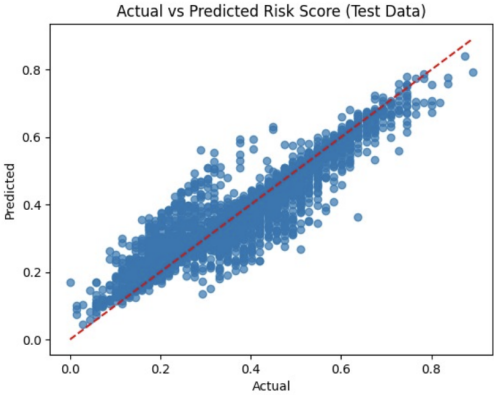
Regression:

Random Forest Regressor

The target variable RiskScore is the last column in the preprocessed datasets. The test RMSE is 0.051.

Best Performing Dataset was with:

- One-Hot Encoding
- Min-Max normalization



Result

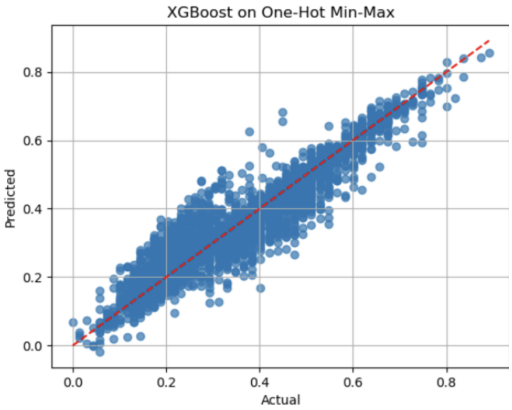
Regression:

XGBoost

Using xgboost's XGBRegressor with `objective='reg:squarederror'`, `max_depth=6`, and `min_child_weight=10`. The test RMSE is 0.046.

Best Performing Dataset was with:

- One-Hot Encoding
- Min-Max normalization



Result

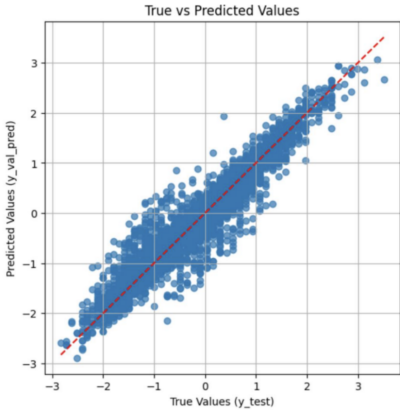
Regression:

Neural Network

The test RMSE is 0.325 for single layer, 0.304 for double layer, and 0.313 for triple layer architecture.

Best Performing Dataset was with:

- One-Hot Encoding
- Z-score normalization



Result

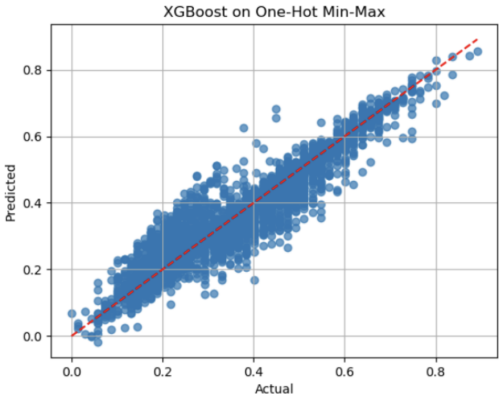
Overall Performance:

XGBoost

For the regression task, one-hot encoded and min-max normalized data showed better results. XGBoost model was the most suitable model with RMSE of 0.046.

Best Performing Dataset was with:

- One-Hot Encoding
- Min-Max normalization



Result

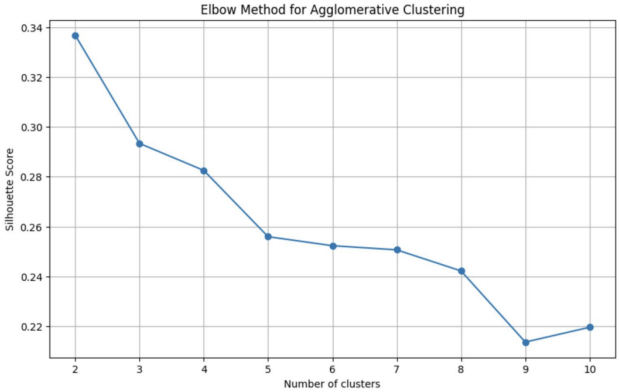
Clustering:

Hierarchical Clustering

As seen in the figure, the elbow occurs at $k = 2$ indicating the optimal number of clusters for hierarchical clustering.

Best Performing Dataset was with:

- One-Hot Encoding
- Z-score normalization



Result

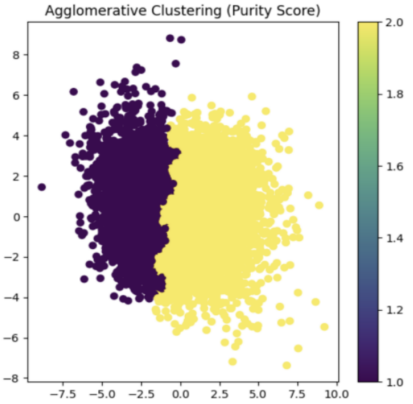
Clustering Visualization:

Hierarchical Clustering

Based on both purity and visual coherence, the ward linkage method performed best. Clusters formed by this method were done so in PCA ($l = 2$).

Best Performing Dataset was with:

- One-Hot Encoding
- Z-score normalization



Result

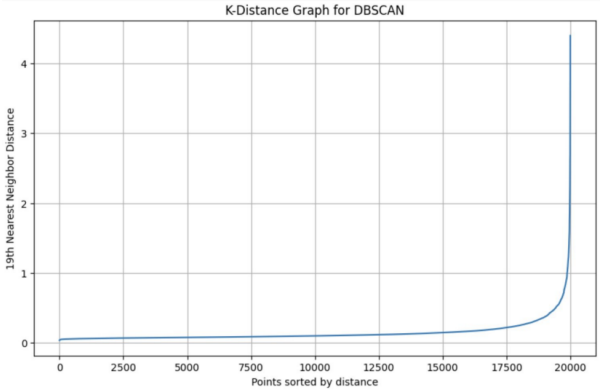
Clustering:

DBSCAN

The “knee” or inflection point in the curve suggests an appropriate eps value. Based on the plot, we selected `eps = 0.5` and `min_samples = 5`.

Best Performing Dataset was with:

- One-Hot Encoding
- Z-score normalization



Result

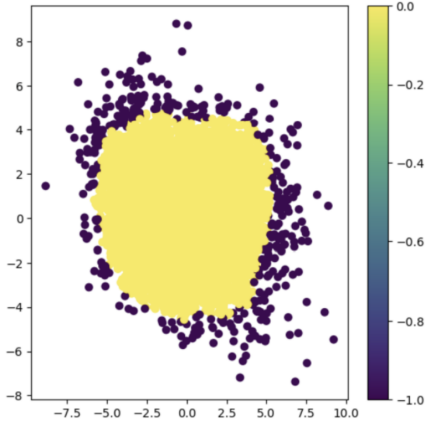
Clustering Visualization:

DBSCAN

The "knee" or inflection point in the curve suggests an appropriate eps value. Based on the plot, we selected `eps = 0.5` and `min_samples = 5`.

Best Performing Dataset was with:

- One-Hot Encoding
- Z-score normalization



Result

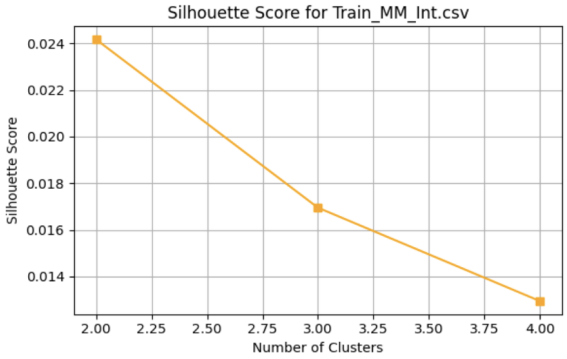
Clustering:

K-Medoid

As seen in the figure, the elbow occurs at $k = 2$ indicating the optimal number of clusters for K-Medoid while using `method="pam"` for efficient calculation.

Best Performing Dataset was with:

- Integer Encoding
- MM normalization



Result

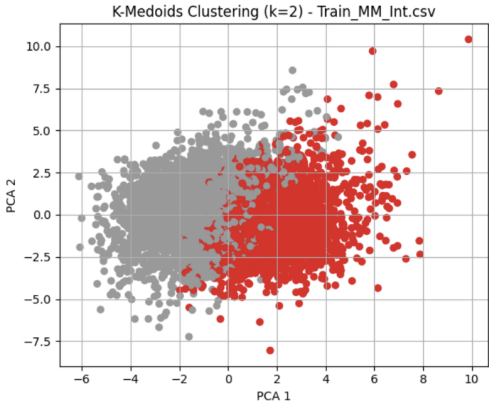
Clustering Visualization:

K-Medoid

As seen in the figure, the elbow occurs at $k = 2$ indicating the optimal number of clusters. Clusters formed by this method were done so in PCA ($l = 2$).

Best Performing Dataset was with:

- Integer Encoding
- MM normalization

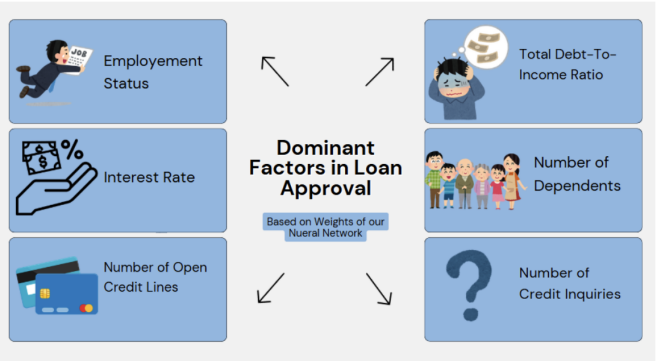


Conclusion

Classification:

LoanApproved

The factors affecting LoanApproved based on the overall best model, i.e. Neural Network.

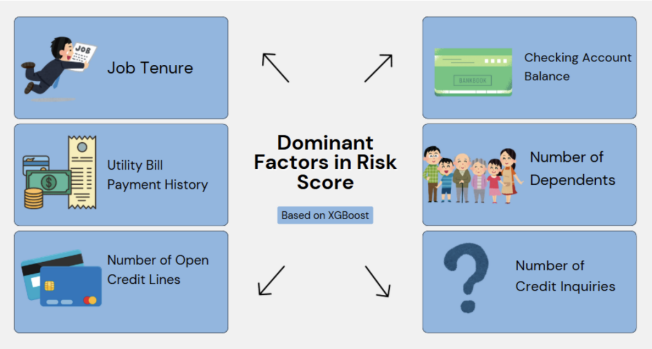


Conclusion

Regression:

RiskScore

The factors affecting RiskScore based on the overall best model, i.e. XGBoost.



Future Scopes

- With our restricted computational power, we were unable to evaluate more complex models; with more complex models our results could improve.
- Hyperparameter tuning was limited.
- Expand to larger, real-world datasets for generalization and real-business impact.

THANK YOU